

PRD — Trustworthy Due Diligence RAG Assistant

1. Project Overview

Project Name

Trustworthy Due Diligence RAG Assistant

Objective

Build an internal AI-powered due diligence assistant capable of answering compliance and due diligence questions based strictly on uploaded enterprise documents.

The system must prioritize:

- Low hallucination risk
- Source-grounded answers
- Traceable citations
- Human review workflow
- Refusal behavior when evidence is insufficient

This is NOT a general-purpose chatbot. This is a retrieval-grounded enterprise compliance assistant.

2. Core Product Principles

Principle 1 — Retrieval Before Generation

The system must retrieve relevant document chunks before calling the LLM.

The LLM must never answer without retrieved evidence.

Principle 2 — System-Enforced Trust

Trustworthiness must be enforced by system architecture, not by prompt instructions alone.

The system must:

- Validate retrieval quality
 - Inject citations programmatically
 - Reject low-confidence responses
 - Route uncertain cases to human review
-

Principle 3 — Prefer Refusal Over Hallucination

If the system cannot confidently answer using retrieved evidence:

- It must refuse to answer
- It must not fabricate policies, clauses, or certifications
- It must route the case to manual review

Business rule:

False negatives are acceptable. False positives are dangerous.

3. Primary Use Cases

Use Case 1 — Compliance Questionnaire Assistance

Example:

Question: "Do you encrypt customer data at rest?"

System behavior:

- Retrieve relevant policy documents
 - Generate grounded answer
 - Provide source citations
 - Output confidence level
-

Use Case 2 — Contract Risk Investigation

Example:

Question: "Is there an unlimited liability clause?"

System behavior:

- Retrieve relevant contract clauses
 - Summarize risk
 - Provide original clause text
 - Show document and page reference
-

Use Case 3 — Internal Policy Search

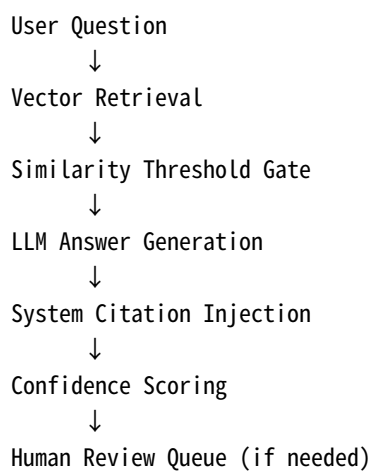
Example:

Question: "What is the company incident response policy?"

System behavior:

- Retrieve policy sections
- Summarize content
- Cite policy source

4. System Architecture



5. Three-Layer Anti-Hallucination Design

Layer 1 — Retrieval Constraints

Goal

Limit what the model can access.

Rules

- Only retrieved chunks are passed into the prompt
- Maximum retrieved chunks: 5
- Retrieval performed using vector similarity search
- Low-similarity queries should bypass LLM generation entirely

Retrieval Parameters

Parameter	Value
Chunk Size	500 characters

Parameter	Value
Chunk Overlap	50 characters
top_k	5
Similarity Threshold	0.70

Layer 2 — Structured Output

Goal

Force deterministic machine-readable outputs.

Required Output Schema

```
{
  "answer": "string",
  "has_source": true,
  "confidence_level": "HIGH",
  "uncertainty_note": "",
  "source_citations": []
}
```

IMPORTANT:

The LLM must NOT generate citations.

Citations must be injected by the backend system directly from retrieval metadata.

Layer 3 — Confidence Validation

Goal

Objectively validate whether retrieved evidence is sufficient.

Validation Logic

Condition	Action
Similarity < 0.70	Reject answer
has_source = false	Human review
High similarity + valid source	Allow answer

6. Citation Grounding Architecture

Critical Design Rule

Citations are system-generated.

The LLM is NOT trusted to generate:

- page numbers
- section numbers
- document names
- quoted text

Correct Citation Flow

```
source_citations = [  
    {  
        "doc_name": chunk["doc_name"],  
        "page": chunk["page"],  
        "quote": chunk["text"][:200]  
    }  
    for chunk in retrieved_chunks  
]
```

The backend injects citations after generation.

7. Recommended Tech Stack

Layer	Technology
Language	Python
Vector DB	Supabase pgvector
Embeddings	OpenAI text-embedding-3-small
LLM	Claude Sonnet
Framework	FastAPI
UI	Streamlit
Document Parsing	PyPDF
ORM	SQLAlchemy

8. Data Model

Document Chunk Schema

```
{
  "chunk_id": "uuid",
  "doc_name": "Security_Policy_v2.pdf",
  "page": 12,
  "text": "All customer data is encrypted...",
  "embedding": "vector",
  "doc_type": "security_policy",
  "version": "v2.1",
  "upload_date": "2026-05-20"
}
```

9. Prompt Design

System Prompt

You are a due diligence and compliance assistant.

STRICT RULES:

1. Only answer using the provided reference documents.
2. Do not use external knowledge.
3. If the documents do not contain sufficient evidence, set has_source to false.
4. Do not fabricate policies, certifications, clauses, or dates.
5. Keep answers concise and evidence-based.

Return valid JSON only.

10. Confidence Scoring Logic

Initial MVP Logic

```
if top_similarity < 0.70:
    return "LOW"

if not model_output["has_source"]:
    return "LOW"

if top_similarity >= 0.85:
```

```
return "HIGH"  
  
return "MEDIUM"
```

11. Human Review Workflow

Review Triggers

Cases enter manual review queue if:

- confidence = LOW
 - has_source = false
 - no retrieval results
 - malformed JSON
-

12. Security Requirements

Requirements

- Uploaded documents must remain internal
 - No training on customer data
 - Audit logging required
 - Access control required in future phases
-

13. Non-Goals (MVP)

The MVP will NOT include:

- AI agents
 - Autonomous workflows
 - Multi-hop reasoning
 - Knowledge graphs
 - Fine-tuned models
 - Multi-user authentication
 - Advanced analytics dashboards
-

14. MVP Success Criteria

The MVP is considered successful if:

- Relevant chunks are retrieved consistently

- Hallucinated citations are eliminated
- Low-confidence answers are rejected
- JSON outputs remain stable
- Users can trace every answer back to source documents

15. Development Roadmap

PHASE 1 — Core Trustworthy RAG Pipeline

Goal

Build a minimal end-to-end trustworthy RAG system.

Scope

- PDF ingestion
- Chunking
- Embedding generation
- Vector storage
- Similarity retrieval
- Threshold gating
- Claude API integration
- JSON output
- System-generated citations
- CLI testing

Deliverables

File Structure

```
project/
|
├── app/
|   ├── ingest.py
|   ├── retrieve.py
|   ├── generate.py
|   ├── confidence.py
|   ├── schemas.py
|   ├── prompts.py
|   └── main.py
|
├── data/
├── docs/
└── tests/
```



```
├── requirements.txt
└── .env
```

Phase 1 File Responsibilities

ingest.py

Responsibilities:

- Parse PDFs
- Split text into chunks
- Generate embeddings
- Store vectors in pgvector

Key functions:

```
load_pdf()
split_chunks()
generate_embedding()
save_chunk()
```

retrieve.py

Responsibilities:

- Embed user query
- Retrieve top-k chunks
- Return similarity scores

Key functions:

```
retrieve_chunks()
filter_low_similarity()
```

generate.py

Responsibilities:

- Build LLM prompt
- Call Claude API
- Parse JSON output

Key functions:

```
build_prompt()
call_claude()
parse_response()
```

confidence.py

Responsibilities:

- Calculate final confidence level
- Apply threshold logic
- Determine review requirement

Key functions:

```
calculate_confidence()
should_escalate()
```

schemas.py

Responsibilities:

- Define Pydantic models
- Validate JSON structure

Key models:

```
AnswerResponse
Citation
Chunk
```

prompts.py

Responsibilities:

- Store system prompts
 - Store JSON output instructions
-

main.py

Responsibilities:

- CLI entry point

- Accept user questions
 - Run full pipeline
 - Print structured output
-

Phase 1 Acceptance Criteria

- User can upload PDFs
 - Retrieval returns relevant chunks
 - LLM outputs valid JSON
 - Citations are backend-generated
 - Low-confidence answers are rejected
 - CLI pipeline works end-to-end
-

PHASE 2 — Retrieval Quality Improvements

Goal

Improve retrieval precision and confidence stability.

Scope

- Metadata filtering
 - avg_top3 similarity scoring
 - Reranker integration
 - Better chunk ranking
 - Improved retrieval evaluation
-

Deliverables

Additional Files

```
app/  
├── reranker.py  
├── metadata_filters.py  
├── ranking.py  
└── evaluation.py
```

Phase 2 File Responsibilities

reranker.py

Responsibilities:

- Re-rank retrieved chunks
- Improve retrieval precision

Suggested models:

- bge-reranker-large
 - Cohere Rerank
-

metadata_filters.py

Responsibilities:

- Filter by doc_type
 - Filter by department
 - Filter by version
-

ranking.py

Responsibilities:

- Compute avg_top3 similarity
 - Compute weighted confidence
 - Rank evidence quality
-

evaluation.py

Responsibilities:

- Measure retrieval accuracy
 - Benchmark retrieval quality
 - Store evaluation metrics
-

Phase 2 Acceptance Criteria

- Retrieval precision improves
 - False positives decrease
 - Similarity scoring becomes more stable
 - Metadata filtering works correctly
-

PHASE 3 — Human Review Interface

Goal

Add enterprise review workflow.

Scope

- Streamlit UI
 - Human review queue
 - Confidence labels
 - Citation display
 - Manual approval workflow
-

Deliverables

Additional Files

```
frontend/  
├── app.py  
├── review_queue.py  
├── components/  
│   ├── answer_card.py  
│   ├── citation_panel.py  
│   └── confidence_badge.py
```

Phase 3 File Responsibilities

app.py

Responsibilities:

- Main Streamlit interface
 - Question submission
 - Display answer results
-

review_queue.py

Responsibilities:

- Display flagged answers
- Manual approve/reject actions
- Human escalation queue

answer_card.py

Responsibilities:

- Render generated answers
 - Display confidence levels
-

citation_panel.py

Responsibilities:

- Show grounded citations
 - Show source quotes
 - Display page references
-

confidence_badge.py

Responsibilities:

- Render HIGH / MEDIUM / LOW indicators
-

Phase 3 Acceptance Criteria

- Users can submit questions through UI
 - Review queue functions correctly
 - Citations are visible and traceable
 - Low-confidence cases are escalated
-

PHASE 4 — Enterprise Readiness (Future)

Goal

Prepare system for production deployment.

Scope

- Authentication
- RBAC
- Audit logging
- API deployment
- Monitoring
- Rate limiting
- Multi-user support

- Document versioning
-

Future Optional Features

- Contract clause extraction
 - Automated risk scoring
 - Due diligence report generation
 - Cross-document comparison
 - Hybrid search
 - Multi-modal document parsing
 - OCR support
-

16. Final Engineering Philosophy

This system should be designed as:

A trustworthy enterprise AI assistant

NOT:

A general-purpose AI chatbot

Core philosophy:

Trust > Fluency
Evidence > Creativity
Refusal > Hallucination